

An Empirical Analysis of Entangled Enhancement in an Autonomous Driving Pipeline of ML Components

Rashedul Arefin Ifty, Fumio Machida

University of Tsukuba

Tsukuba, Japan

ifty.rashedul@sd.cs.tsukuba.ac.jp, machida@cs.tsukuba.ac.jp

Abstract—Entangled enhancement is a retraining that improves one component of a machine learning system (MLS) but fails to improve the whole system. This paper measures entangled enhancement in an autonomous driving pipeline in which the perception, prediction and planning modules are all machine learning (ML) components. The perception module is updated 30 times on data from a new city, and every update improves the perception module. The prediction and planning modules are trained once, either on the ground truth or on the output of the old perception module, and stay fixed. When the prediction module was trained on the output of the old perception module, the prediction error increased in 23 of the 30 updates, in 11 by more than random variation and in 3 with statistical significance. When the prediction module was trained on the ground truth, the counts were 5, 0 and 0. The planning error changed by at most 0.13 m. No measure of output change that needs no ground truth predicted which updates harmed the prediction module.

Index Terms—autonomous driving, entangled enhancement, machine learning system, model maintenance, retraining

I. INTRODUCTION

An MLS is often built as a pipeline, which is a set of ML components connected in a line, where each component uses the output of the preceding component. A component that comes earlier in the line is upstream of a component that comes later, and the later component is downstream. Retraining an upstream component can improve that component and still fail to improve the whole system. Wang and Machida call this effect entangled enhancement and model when an MLS should retrain each component [1], [2]. The models of Wang and Machida need the probability that retraining the upstream component leaves the downstream component working. This probability has not been measured on a real system.

A driving pipeline has three components. The perception module finds the objects around the ego vehicle, which is the vehicle that the pipeline drives. The prediction module predicts where each object will move, and the planning module chooses the path of the ego vehicle. The three modules are trained separately and are updated one at a time. An earlier experiment in this study retrained the perception module of such a pipeline 15 times. In 6 of the 15 updates, the planned path moved further from the path of the human driver. The prediction module and the planning module in that pipeline were fixed rules. Fixed rules cannot adapt to the mistakes of

the perception module. That experiment therefore could not test the adaptation that the theory of entangled enhancement describes.

This paper replaces both fixed rules with ML components. The ground truth is the set of object positions labelled by hand. The prediction module and the planning module are trained in two ways, once on the ground truth and once on the output of the old perception module. Only the second way allows the downstream modules to adapt to the old mistakes. The comparison of the two ways therefore shows whether a harm after an update comes from that adaptation.

This paper makes three contributions.

- 1) A driving pipeline of three ML components, trained on a reduced version of the nuScenes dataset [3] and checked against simple baselines.
- 2) A series of 30 updates of the perception module, with the number of harmful updates counted at three levels of evidence.
- 3) A test of nine measures of output change that need no ground truth, as a possible symptom of entangled enhancement.

II. BACKGROUND AND RELATED WORK

This section summarises the earlier work that the experiment builds on. Sculley et al. [4] describe how a change to one component of an MLS changes the behaviour of every component that uses the changed output. Wu et al. [5] show that improving one model of a system can make the whole system worse. A harm of this kind can have two causes. In the first cause, the downstream components adapted during training to the mistakes of the old upstream component and are disturbed when those mistakes change. In the second cause, the upstream component improves on average but becomes worse on the cases that matter downstream.

AutoBot-Ego [6] predicts the future of one target object from the past positions of the surrounding objects, and PlanT [7] plans the path of the ego vehicle from a list of objects. Weng et al. [8] show that a prediction module can learn by attention which of the objects found in different images are the same object. The prediction error increases when the input comes from a real perception module instead of the ground truth [9]. A planning module that receives the

state of the ego vehicle can score well on nuScenes while ignoring the objects [10], [11].

Rabanser et al. [12] detect a change in the data seen by a model without ground truth from distances between distributions, where a distribution is the pattern of values that an output takes over many inputs. This paper applies such distances to the outputs of the perception module and of the prediction module as a candidate symptom of entangled enhancement.

III. METHODOLOGY

This section defines the notation and states the two hypotheses. The section then describes the workflow of the experiment and the procedure that produces the updates, which together determine how each hypothesis is tested.

A. Pipeline and Notation

The perception module C_1 finds the objects in the image of the front camera. The prediction module C_2 predicts the path of each object over the next 3 s, and the planning module C_3 chooses the path of the ego vehicle over the same 3 s. An update is one retraining of C_1 on data from a new city. We call C_1 before an update the old C_1 and C_1 after the update the new C_1 .

We write δ_1 for the gain of C_1 in mAP50, the standard accuracy measure for object detection. The prediction error is the minADE, which is the average distance between the predicted path and the true path of an object, taken for the best of the paths that C_2 predicts. The planning error is the distance between the planned position and the position of the human driver at 3 s. We write Δ_2 and Δ_3 for the increase of the prediction error and of the planning error after an update. An update shows entangled enhancement when

$$\delta_1 > 0 \quad \text{and} \quad (\Delta_2 > 0 \text{ or } \Delta_3 > 0). \quad (1)$$

B. Hypotheses

H1 (existence): An update of C_1 can increase the prediction error or the planning error of a pipeline of three ML components, even when the update makes C_1 more accurate. The increase is larger when C_2 and C_3 were trained on the output of the old C_1 than when C_2 and C_3 were trained on the ground truth.

H2 (symptom): The change in the output of C_1 and C_2 after an update, measured without ground truth, is larger for the updates that meet condition (1) than for the other updates. The gain δ_1 alone does not separate the two groups. If H2 holds, an engineer can compute the change before using an update and retrain C_2 and C_3 when the change is large.

C. Workflow

Fig. 1 shows the six stages of the experiment. The data are split into four parts. Boston 1 and Boston 2 train the modules, Singapore 1 provides the data for the updates, and Singapore 2 is the test set. The old C_1 is trained with two random seeds, where a random seed is the starting number for the random choices made during training. C_2 and C_3 are trained once

Algorithm 1: Update procedure for one old C_1

Input : old C_1^{old} , settings $\mathcal{S}$, Boston 2, Singapore 1, Singapore 2
Output: δ_1 , Δ_2 , Δ_3 and evidence level per update and route
 Train C_2^A, C_3^A on the ground truth of Boston 2
 Train C_2^B, C_3^B on the output of C_1^{old} on Boston 2
foreach route $r \in \{A, B\}$ **do**
 $E_2^r, E_3^r \leftarrow$ errors of $C_1^{\text{old}} \rightarrow C_2^r \rightarrow C_3^r$ on Singapore 2
 $\tau_2, \tau_3 \leftarrow$ random variation from the other old C_1
foreach setting $s \in \mathcal{S}$ **do**
 $C_1^{\text{new}} \leftarrow$ retrain C_1^{old} on Singapore 1 with s
 $\delta_1 \leftarrow \text{mAP50}(C_1^{\text{new}}) - \text{mAP50}(C_1^{\text{old}})$
 foreach route $r \in \{A, B\}$ **do**
 $\Delta_2, \Delta_3 \leftarrow$ errors of $C_1^{\text{new}} \rightarrow C_2^r \rightarrow C_3^r$ minus E_2^r, E_3^r
 record the three levels of evidence
 record the output change of C_1 and C_2 for H2

along two routes. Route A uses the ground truth, and route B uses the output of the old C_1 . Each of the 30 updates is then tested in the full pipeline with C_2 and C_3 kept fixed.

D. Update Procedure

Algorithm 1 gives the procedure for one old C_1 . The random variation τ_2 and τ_3 is the largest change of the errors when the old C_1 is replaced by the other old C_1 , which differs only in the random seed. The first level of evidence counts any increase, and the second level counts an increase above the random variation. The third level counts an increase whose 95 % bootstrap interval lies above zero. The bootstrap repeats the measurement on test sets drawn with replacement from the test scenes.

IV. EXPERIMENTAL SETUP

This section gives the data, the updates of C_1 , the ML models for C_2 and C_3 , and the measurements. All models were trained on one Tesla T4 graphics card.

A. Data

The experiment uses 350 of the 850 scenes of nuScenes [3], which are driving clips of about 20 s from Boston and Singapore, with the full-size images of the front camera only (4.6 GB). Boston 1 (100 scenes) trains C_1 , and Boston 2 (100 scenes) trains C_2 and C_3 , with 14 of the scenes kept for validation. A perception module is much more accurate on the scenes used to train that module. Separate scenes give C_2 and C_3 the realistic mistakes of C_1 [9]. Singapore 1 (100 scenes) is used for the updates, and Singapore 2 (50 scenes, 2,020 images) is used for testing. A keyframe is an image with labelled objects, recorded twice per second. The test set has 1,720 test samples, one for every keyframe with 3 s of future.

B. Updates of the Perception Module

C_1 is YOLOv11n [13], trained for 10 passes over Boston 1 with the random seeds 0 and 1. The two old C_1 reach an mAP50 of 0.109 and 0.117 on Singapore 2. An update retrains an old C_1 on Singapore 1 with one of 15 settings. Nine settings use 100 %, 50 % or 25 % of the images (3,575, 1,788 or 894) with the seeds 1, 2 and 3 and 10 passes. Six settings use all

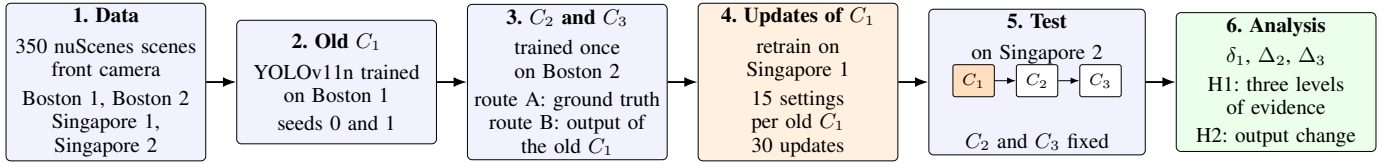


Fig. 1. Workflow of the experiment

images with 5 or 20 passes and the same seeds. The 15 settings applied to both old C_1 give 30 updates. The test scenes are never used to choose a model.

C. ML Prediction and Planning Modules

C_2 follows AutoBot-Ego [6]. For each current object, a transformer reads the objects of the last four keyframes relative to that object and outputs six possible paths of 3 s, without a hand-written rule that links objects across images [8]. C_3 follows PlanT [7]. Every object is one input with the current position, the most likely path from C_2 , the class and the confidence. Two further inputs hold the driving command and the path of the ego vehicle over the last 1.5 s. The command is left, straight or right, from the lateral position of the human path at 3 s [10]. A four-layer encoder and a recurrent decoder output six points of the planned path.

Both models were trained on Boston 2 with early stopping on the validation scenes and with mirror and rotation augmentation. An input of the ego path can let a planning module ignore the objects [11], and the experiment checks whether C_3 uses the objects. Removing the objects from the input of C_3 moves the planned position at 3 s by 0.73 to 1.65 m on average, which is above the threshold of 0.5 m fixed before the experiment.

D. Training Routes and Measurements

C_2 and C_3 are trained once along route A and once along route B for each old C_1 , and the trained models stay fixed during all updates. The prediction error is computed over the objects found by C_1 that are matched one to one with a ground truth object of the same class. The collision rate is the fraction of test samples in which the planned path overlaps an object. The outputs of C_2 and C_3 on the ground truth stayed exactly the same before and after every update, which confirms that only the output of C_1 changed. The random variation is $\tau_2 = 0.112\text{m}$ and $\tau_3 = 0.018\text{m}$. The bootstrap draws the 50 test scenes 2,000 times with replacement.

V. RESULTS

This section first checks that C_2 and C_3 work, because a count of harmful updates is meaningful only for working modules. The section then reports the effect of the updates and the test of the symptom. Every update made C_1 more accurate, with δ_1 between 0.048 and 0.118.

TABLE I
ML COMPONENTS AGAINST SIMPLE BASELINES ON THE TEST SET (M)

Model and input	Model	Baseline
C_2 on the ground truth	0.77	4.48 (stays still)
C_2 on the output of C_1	2.39 to 2.95	5.39 to 5.85 (stays still)
C_3 without ego input	9.3 to 12.5	9.25 (average path)
C_3 with the ego path	3.70 to 4.16	9.25 (average path)

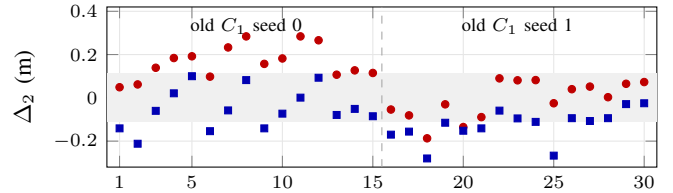


Fig. 2. Δ_2 in the 30 updates. Red circles show route B, blue squares show route A, and the grey band shows the random variation.

A. Validity of the ML Components

Table I compares C_2 and C_3 with simple baselines on the test set. C_2 beats a baseline in which every object stays where the object is, and on the ground truth C_2 also beats a constant-velocity extrapolation of 1.01 m. The variant of C_3 without any ego input does not beat the baseline that drives the average Boston path for each command. The experiment therefore uses the variant with the ego path, which has a collision rate of 0.05 to 0.08 against 0.18 for the baseline.

B. Prediction Module

Fig. 2 shows Δ_2 for every update and both routes. In all 30 updates, Δ_2 of route B is larger than Δ_2 of route A. The difference is largest for the updates from the old C_1 with seed 0, where Δ_2 of route B lies between +0.049 and +0.284 m with a mean of +0.165 m.

Table II counts the updates with an increase at the three levels of evidence. Above random variation, the prediction error increased in 11 updates along route B and in none along route A. All 11 updates start from the old C_1 with seed 0. With 50 test scenes, the bootstrap interval of Δ_2 spans about $\pm 0.25\text{m}$. Only 3 updates along route B and none along route A have an interval above zero.

C. Planning Module

The planning error changed little. The mean Δ_3 lies between -0.006 and $+0.058\text{m}$, and the largest value is 0.13 m against a planning error of about 4 m. The increase appears

TABLE II
UPDATES, OUT OF 30, WITH AN INCREASE OF THE ERROR

Level of evidence	Prediction Δ_2		Planning Δ_3	
	B	A	B	A
Any increase	23	5	18	29
Above random variation	11	0	13	29
Bootstrap interval above zero	3	0	1	10

more often along route A than along route B at every level in Table II. Adaptation to the mistakes of the old C_1 cannot explain an increase along route A.

D. Retraining on the New Perception Module

Retraining C_2 and C_3 with C_1 can remove the harm. Two further updates, one from each old C_1 with all images and 10 passes, were used to retrain C_2 and C_3 on the output of the new C_1 . Without retraining, the prediction error after the update is 2.75 and 2.61 m, and the planning error is 4.14 and 4.00 m. With retraining, the prediction error is 2.39 and 2.45 m, and the planning error is 3.71 and 3.70 m.

E. Symptom of Entangled Enhancement

Nine measures of output change need no ground truth, seven on the output of C_1 and two on the output of C_2 . The rank correlation with Δ_2 is close to 1 when a larger measure always comes with a larger Δ_2 . The AUROC is the probability that a harmful update ($\Delta_2 > 0$) has a larger measure than a harmless update. Over all 30 updates, the shift of the predicted position of the same object has the highest rank correlation (+0.68, AUROC 0.81). For each old C_1 separately, the rank correlation is only +0.29 and -0.28. The high value over all updates comes from the difference between the two old C_1 . The gain δ_1 has a rank correlation of +0.04. No measure separates harmful and harmless updates consistently, and H2 is not supported.

VI. DISCUSSION

This section discusses three limits of the results and the experiment that can remove each limit.

Test set size: With 50 test scenes, the bootstrap interval of a single update is wide, and only 3 of the 30 updates are significant individually. The pattern across updates is stronger, with route B worse than route A in all 30 updates, although the 30 results share two starting models and one test set and are not independent.

Starting model: All 11 updates above random variation start from the old C_1 with seed 0, and the updates from seed 1 have a mean Δ_2 of -0.008 m. Two starting models cannot show how often a starting model leads to harm. More old C_1 trained with other seeds are needed for that estimate.

Scope: All results come from one pair of models for C_2 and C_3 , from the front camera only and without maps. The objects move the planned position of C_3 by only 0.73 to 1.65 m, which may explain the small values of Δ_3 . Other pairs of models can show whether the result holds in other pipelines.

VII. CONCLUSION

This paper tested entangled enhancement in a driving pipeline of three ML components with 30 updates of the perception module. The prediction module trained on the output of the old perception module became worse more often and by more than the prediction module trained on the ground truth. This result supports H1 for the prediction module and points to adaptation to the mistakes of the old perception module as the cause. The planning error changed little. No measure of output change that needs no ground truth predicted the harm, and H2 is not supported. Work in progress develops a method for the early detection of entangled enhancement, which flags a harmful update of the perception module before the update is used and before ground truth for the new data exists.

REFERENCES

- [1] Z. Wang and F. Machida, "Maintaining performance of a machine learning system against imperfect retraining," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 1782–1787.
- [2] Z. Wang and F. Machida, "Exploiting the availability-continuity trade-off in imperfect retraining of machine learning systems," in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 406–417.
- [3] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuScenes: A multimodal dataset for autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 621–11 631.
- [4] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 2503–2511.
- [5] R. Wu, C. Guo, A. Hannun, and L. van der Maaten, "Fixes that fail: Self-defeating improvements in machine-learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 11 745–11 756.
- [6] R. Girgis, F. Golemo, F. Codevilla, M. Weiss, J. A. D'Souza, S. E. Kahou, F. Heide, and C. Pal, "Latent variable sequential set transformers for joint multi-agent motion prediction," in *International Conference on Learning Representations (ICLR)*, 2022.
- [7] K. Renz, K. Chitta, O.-B. Mercea, A. S. Koepke, Z. Akata, and A. Geiger, "PlanT: Explainable planning transformers via object-level representations," in *Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 205, 2022.
- [8] X. Weng, B. Ivanovic, K. Kitani, and M. Pavone, "Whose track is it anyway? Improving robustness to tracking errors with affinity-based trajectory prediction," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [9] Y. Xu, L. Chambon, É. Zablocki, M. Chen, A. Alahi, M. Cord, and P. Pérez, "Towards motion forecasting with real-world perception inputs: Are end-to-end approaches competitive?" in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 18 428–18 435.
- [10] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, "Rethinking the open-loop evaluation of end-to-end autonomous driving in nuScenes," *arXiv preprint arXiv:2305.10430*, 2023.
- [11] Z. Li, Z. Yu, S. Lan, J. Li, J. Kautz, T. Lu, and J. M. Alvarez, "Is ego status all you need for open-loop end-to-end autonomous driving?" in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [12] S. Rabanser, S. Günemann, and Z. C. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [13] G. Jocher and J. Qiu, "Ultralytics YOLO11," <https://github.com/ultralytics/ultralytics>, 2024.